

LP-II: Web Application Development

Assignment on *Yargs* Package

Q1) Illustrate the need of “*yargs*” package.

Ans: In Node.js, command-line applications often need to accept and process user input from the terminal. While it's possible to do this using `process.argv`, handling and organizing arguments manually can get messy. This is where the *yargs* package becomes extremely useful.

1. The Problem with `process.argv`

By default, Node.js gives access to command-line arguments through `process.argv`. However:

- It's just a raw array.
- We have to manually parse and extract values.
- There's no built-in support for flags, default values, or help messages.
- Adding validations and error handling requires extra code.

This can lead to unreadable and error-prone code, especially in larger scripts or tools.

2. How *yargs* Solves This

The *yargs* package is built specifically to address these issues. It helps by:

- ❖ Parsing arguments automatically and presenting them as a neat JavaScript object.
- ❖ Allowing easy creation of flags, options, and commands.
- ❖ Supporting default values, aliases, and input validation.
- ❖ Auto-generating help and usage messages, improving user experience.

Providing meaningful error messages for missing or incorrect input.

3. Simple Example

Here's a quick example to show how easy *yargs* makes it:

```
const yargs = require('yargs');  
const argv = yargs.argv;  
console.log(`Hello, ${argv.name}!`);
```

If we run this script like:

```
node app.js --name=John
```

The output will be:

```
Hello, John!
```

Without *yargs*, doing the same would require much more manual parsing.

4. Why It's Needed

To sum up, yargs is needed because it:

- **Saves time** by reducing the need for manual parsing.
- **Keeps code clean and organized.**
- **Improves the usability** of command-line tools.
- Helps follow **best practices** when interacting with users through the terminal.

In real-world projects or scripting tasks, using yargs can turn a basic script into a polished and professional command-line tool with minimal effort.

5. Conclusion

Instead of reinventing the wheel with `process.argv`, yargs offers a powerful, ready-made solution for handling command-line input in Node.js applications. For any developer working with CLI tools, this package is practically a must-have.

Q2) Implement small application which will make use of mentioned package (I/p, O/p)

Ans: Here's a **simple calculator** built using **yargs**. This CLI app lets you perform basic arithmetic operations: add, subtract, multiply, and divide.

App.js:

```
const yargs = require('yargs');

// Define the "calculate" command
yargs.command({
  command: 'calc',
  describe: 'Perform basic arithmetic operations',
  builder: {
    a: {
      describe: 'First number',
      demandOption: true,
      type: 'number',
    },
    b: {
      describe: 'Second number',
      demandOption: true,
      type: 'number',
    },
    op: {
      describe: 'Operation (add, sub, mul, div)',
      demandOption: true,
      type: 'string',
    },
  },
```

```
    },
    handler(argv) {
        const { a, b, op } = argv;

        // Perform operation based on 'op' value
        switch (op) {
            case 'add':
                console.log(`Result: ${a + b}`);
                break;
            case 'sub':
                console.log(`Result: ${a - b}`);
                break;
            case 'mul':
                console.log(`Result: ${a * b}`);
                break;
            case 'div':
                if (b === 0) {
                    console.log('Error: Cannot divide by zero.');
```

} else {

```
                console.log(`Result: ${a / b}`);
            }
            break;
            default:
                console.log('Invalid operation. Use: add, sub, mul, or div.');
```

}

```
    },
    });
});
```

```
// Parse the input arguments
```

```
yargs.parse();
```

OUTPUT:

```
PS C:\Users\Yash\Desktop\Yargs> node app.js calc --a=10 --b=5 --op=add
```

```
Result: 15
```

```
PS C:\Users\Yash\Desktop\Yargs> node app.js calc --a=10 --b=5 --op=sub
```

```
Result: 5
```

```
PS C:\Users\Yash\Desktop\Yargs> node app.js calc --a=10 --b=5 --op=mul
```

```
Result: 50
```

```
PS C:\Users\Yash\Desktop\Yargs> node app.js calc --a=10 --b=5 --op=div
```

```
Result: 2
```

Q3) Illustrate the need for code of ethics:

Ans: The Need for a Code of Ethics in Web Application Development

A **Code of Ethics** is crucial in web application development to ensure that the technology we build is safe, accessible, and respects user privacy. In an era where web applications are central to business, education, and personal life, the ethical guidelines governing their development help safeguard users and maintain trust.

1. Guiding Developer Behavior

A Code of Ethics provides web developers with clear standards on what constitutes responsible and ethical behavior. It helps developers navigate complex situations, such as balancing innovation with user security or making decisions that prioritize usability over convenience or profit.

2. Protecting User Privacy and Security

One of the most critical aspects of web application development is ensuring the safety and privacy of user data. Developers must adhere to ethical practices to safeguard sensitive information, preventing misuse, data breaches, or unauthorized access. A Code of Ethics ensures that these practices are not overlooked in the pursuit of efficiency or profitability.

3. Promoting Accessibility

Web developers are responsible for ensuring their applications are accessible to everyone, including people with disabilities. Ethical guidelines in web development emphasize the importance of creating applications that comply with accessibility standards (like WCAG) to ensure all users can interact with the web product.

4. Building User Trust

Web applications handle sensitive tasks like online banking, shopping, and communication. Following an ethical code helps establish transparency, build trust, and prevent unethical practices such as exploiting user data or misleading users. Trust is a key factor in user retention, especially for businesses that rely on their web platforms.

5. Encouraging Accountability in Development

A Code of Ethics encourages developers to be accountable for their work, from testing to deployment. This accountability ensures that developers take responsibility for bugs, security flaws, or design issues, which is vital for maintaining a high standard in web development and ensuring the application performs as expected.